

Início Rápido do AI-DLC

Author: Raja SP, AWS Labs

Source: <https://github.com/awslabs/aidlc-workflows>

Adaptation: Ricardo de Luna Galdino, EngSoft Learn

Download: [inputs-quickstart.pt-br.pdf](#)

O AI-DLC (AI-Driven Development Life Cycle) é um workflow estruturado que guia um assistente de IA no planejamento, design e construção de software. Antes de iniciar um projeto, você fornece dois documentos que dizem à IA **o que construir e quais ferramentas usar**.

O Que Você Precisa Fornecer

1. Documento de Visão — o que construir e por quê

Seção	O Que Escrever	Extensão
Resumo Executivo	Um parágrafo: o que é, para quem é, por que é importante	3-5 frases
Problema	O problema de negócio específico que isso resolve	1-2 parágrafos
Usuários-Alvo	Quem vai usar, o que cada tipo de usuário precisa	Uma tabela com uma linha por tipo de usuário
Métricas de Sucesso	Como você mede se este projeto teve sucesso	Uma tabela com metas mensuráveis
Visão do Escopo Completo	Tudo que o produto poderia se tornar na maturidade, organizado por área	Quantas áreas forem necessárias
Escopo MVP — Funcionalidades DENTRO	Cada funcionalidade incluída no primeiro release, com justificativa	Uma tabela. Se não estiver listado, não está no MVP.
Escopo MVP — Funcionalidades FORA	Funcionalidades deliberadamente excluídas do MVP, com motivo e fase alvo	Uma tabela. Isso previne o aumento de escopo.
Riscos e Perguntas Abertas	O que pode dar errado, o que ainda não está decidido	Tabelas e listas com marcadores

Princípio chave: Separe a visão completa do MVP. A visão completa é aspiracional. O MVP é a menor coisa que entrega valor.

Guia completo: [vision-document-guide.pt-br.md](https://github.com/mauricioferreira/vision-document-guide.pt-br.md)

Exemplo prático: [example-vision-scientific-calculator-api.pt-br.md](https://github.com/mauricioferreira/example-vision-scientific-calculator-api.pt-br.md)

2. Documento de Ambiente Técnico — quais ferramentas usar

Seção	O Que Escrever	Extensão
Linguagens	Linguagens obrigatórias, permitidas e proibidas com versões	Uma tabela por categoria
Frameworks e Bibliotecas	Obrigatórios, preferidos e proibidos com justificativa e alternativas	Uma tabela por categoria
Serviços Cloud	Lista de permissões e lista de bloqueios de serviços cloud com restrições	Uma tabela por lista
Arquitetura e Padrões	Estilo de API, padrões de dados, mensageria, estrutura do projeto	Seções curtas com tabelas
Segurança	Método de autenticação, criptografia, validação de entrada, gerenciamento de segredos e um framework de conformidade de segurança com controles documentados por categoria	Várias subseções
Testes	Tipos de teste, metas de cobertura, ferramentas, gates de CI/CD	Tabelas
Código de Exemplo	Código modelo mostrando padrões canônicos para endpoints, funções, testes e infraestrutura	Arquivos de código funcionais em um diretório <code>examples/</code>

Princípio chave: Seja explícito sobre o que é permitido e o que não é. Listas de permissão e bloqueio evitam que a IA faça suposições.

Guia completo: [technical-environment-guide.pt-br.md](https://github.com/pt-br/technical-environment-guide)

Exemplo prático: [example-tech-env-scientific-calculator-api.pt-br.md](https://github.com/pt-br/example-tech-env-scientific-calculator-api)

Input Mínimo Viável

Se quiser começar rápido e preencher detalhes depois, forneça pelo menos isso:

Visão (mínimo)

1. Um parágrafo dizendo o que você está construindo e para quem
2. Uma lista de funcionalidades do MVP (o que está NO escopo)
3. Uma lista do que NÃO está no MVP
4. Perguntas abertas -- coisas que você já sabe que são incertas ou não resolvidas

Perguntas abertas são opcionais, mas valiosas. Elas alimentam diretamente a Análise de Requisitos como ambiguidades pré-declaradas, de modo que o AI-DLC as aborda cedo em vez de fazê-las surgir como surpresas no meio do design.

Veja example-minimal-vision-scientific-calculator-api.pt-br.md para um exemplo prático.

Ambiente Técnico (mínimo)

1. Linguagem e versão
2. Gerenciador de pacotes
3. Framework web (se aplicável)
4. Provedor cloud e modelo de deploy (ou "apenas local")
5. Framework de testes
6. Bibliotecas e serviços proibidos -- use uma tabela: proibido | motivo | use ao invés disso
7. Requisitos básicos de segurança (método de autenticação, abordagem de validação de entrada, gerenciamento de segredos)
8. Padrões de código de exemplo -- um exemplo curto de cada para um endpoint típico, função e teste

Sobre o item 6: incluir o motivo e a alternativa recomendada é importante. Sem eles, o AI-DLC pode honrar a proibição mas não entender a intenção bem o suficiente para tomar boas decisões de substituição.

Sobre o item 8: mesmo um ou dois exemplos curtos dão ao AI-DLC um padrão concreto a seguir durante a geração de código em vez de inventar o seu próprio. Esta é a adição de maior impacto além do básico.

Veja example-minimal-tech-env-scientific-calculator-api.pt-br.md para um exemplo prático de ambos.

Todo o resto pode ser respondido através das perguntas de esclarecimento do AI-DLC durante a fase Inception. Quanto mais você fornecer antecipadamente, menos perguntas a IA precisará fazer.

Projetos Brownfield

Se você está adicionando ou modificando uma codebase existente, seus inputs precisam responder a um conjunto diferente de perguntas. Os guias completos cobrem brownfield em detalhes, mas o mínimo é:

Visão (mínimo brownfield)

1. Estado atual -- um parágrafo descrevendo o que o sistema faz hoje
2. O que estamos adicionando ou mudando -- uma descrição clara da mudança
3. Funcionalidades NO escopo para esta iteração
4. Funcionalidades FORA do escopo para esta iteração
5. O que NÃO DEVE mudar -- componentes existentes, APIs ou dados que o novo trabalho não deve tocar
6. Perguntas abertas

A seção "o que não deve mudar" é fundamental. O AI-DLC executará uma etapa de Engenharia Reversa para analisar sua codebase existente, mas ser explícito sobre os limites evita que ele proponha mudanças que desestabilizariam partes funcionais do sistema.

Veja [example-minimal-vision-brownfield.pt-br.md](#) para um exemplo prático.

Ambiente Técnico (mínimo brownfield)

1. Stack existente -- linguagem, framework, banco de dados, infra -- com versões
2. O que adicionar (novos serviços, tabelas, componentes)
3. O que deve permanecer inalterado -- serviços, schemas, contratos, configs que não devem ser tocados
4. Padrões proibidos -- bibliotecas ou abordagens que conflitam com a codebase existente
5. Requisitos básicos de segurança -- como autenticação e segredos funcionam no sistema existente
6. Padrões de código de exemplo da codebase existente

Os padrões de código de exemplo são especialmente importantes para brownfield. O AI-DLC deve gerar código que pareça que pertence à codebase existente, não código que introduz novas convenções ao lado das antigas. Tire seus exemplos de arquivos existentes reais.

Veja [example-minimal-tech-env-brownfield.pt-br.md](#) para um exemplo prático.

O Que Acontece Depois que Você Fornece Esses Documentos

O AI-DLC passa por duas fases principais:

Inception — entender e planejar

1. Detecta seu workspace (novo projeto ou código existente)
2. Analisa requisitos (faz perguntas de esclarecimento se algo não estiver claro)
3. Cria user stories (se o projeto justificar)
4. Constrói um plano de execução (quais etapas executar, quais ignorar)
5. Projeta componentes e unidades de trabalho (se a complexidade justificar)

Construction — projetar e construir (por unidade de trabalho)

1. Design funcional (lógica de negócio, modelos de domínio)
2. Requisitos e design de NFR (performance, segurança, escalabilidade)
3. Design de infraestrutura (mapeia para serviços cloud reais)
4. Geração de código (escreve o código, testes e artefatos de deploy)
5. Build e teste (instruções de build, execução de testes, verificação)

Cada etapa requer sua aprovação antes de prosseguir. Você pode solicitar mudanças, adicionar etapas ignoradas ou redirecionar em qualquer gate.

Visão Geral dos Arquivos

```
docs/writing-inputs/
  inputs-quickstart.pt-br.md          <-- Você está aqui
  vision-document-guide.pt-br.md      <-- Como escrever um documento de
visão
  technical-environment-guide.pt-br.md <-- Como escrever um documento de
ambiente técnico

-- Exemplos greenfield (novo projeto do zero) --
example-vision-scientific-calculator-api.pt-br.md <-- Exemplo completo: visão CalcE
ngine
example-tech-env-scientific-calculator-api.pt-br.md <-- Exemplo completo: ambiente té
cnico CalcEngine
example-minimal-vision-scientific-calculator-api.pt-br.md<-- Exemplo mínimo: visão CalcEng
ine
example-minimal-tech-env-scientific-calculator-api.pt-br.md<-- Exemplo mínimo: ambiente té
cnico CalcEngine

-- Exemplos brownfield (adicionando a um sistema existente) --
example-minimal-vision-brownfield.pt-br.md          <-- Exemplo mínimo: módulo de dev
oluções em plataforma existente
example-minimal-tech-env-brownfield.pt-br.md        <-- Exemplo mínimo: módulo de dev
oluções em plataforma existente
```